PROJECT GUIDE

Healthcare Intelligence on Snowflake

Data architecture · Semantic view · Cortex Search · Cortex Analyst
Cortex Agent · Security · Streamlit app

What we built, why we built it that way, and what to keep in mind.

How to use this guide

Read it top to bottom once, then use the checklists before you build, demo or interview. Every section has the same shape: **what it is**, **why we use it**, **how we did it in this project**, and the **important points** to remember. Coloured boxes mark key ideas, things to watch out for, and what we actually did.

#	Section
1	The project in one page
2	Data architecture: the layers and why
3	The build, file by file
4	Semantic modelling, model, view and layer
5	The semantic view in detail + checklist
6	Cortex Search
7	Cortex Analyst
8	Cortex Agent + checklist
9	Agent skills
10	Evaluation
11	Security and governance
12	Performance: marts, clustering, caching
13	The Streamlit app + checklist
14	Problems we hit and how we fixed them
15	Interview quick answers
16	Glossary

1. The project in one page

Goal: let anyone in a healthcare organisation ask questions in plain English, such as "*How many patients have had a heart attack?*", and get an answer that is **correct**, **consistent** with the dashboard, and **safe** (no personal data leaks).

The problem with raw data

- **Cryptic names:** columns such as `ENCOUNTERCLASS` or `STATUS1` mean nothing to a business user or to an AI.
- **Clinical vocabulary:** people say "heart attack", but the data says "Myocardial infarction (disorder)". A naive filter on "heart attack" returns **0**.
- **Size:** 763 million clinical measurements. Asking for "the latest BMI per patient" from raw rows every time is slow and easy to get wrong.
- **Personal data:** names, SSNs, addresses and birth dates must be protected by role.
- **Different answers:** if every analyst writes their own SQL, two people get two different readmission rates.

What we built

Piece	One-line description
Business views	Rename raw columns to business names; mask personal data by role
Semantic view	One business dictionary: tables, joins, definitions, metrics, verified questions
Cortex Search	Translates everyday words into the exact clinical terms stored in the data
Cortex Agent	Answers questions by using Cortex Search + Cortex Analyst (text-to-SQL)
KPI marts	Small summary tables, built from the semantic view, for a fast dashboard
Streamlit app	Dashboard + AI chat + patient lookup + PDF report, all controlled by role
Governance	Masking policies, role permissions, audit log, evaluation (13/13 passed)

Key numbers (synthetic Synthea data)

Measure	Value
Patients	1.42mn (1,421,656)
Visits	64.54mn (64,535,917)
Clinical measurements	about 763mn rows
Total claim amount	$174.25bn
Patients who had a heart attack	61.82K (61,820)
30-day readmission rate	17.06%
Agent evaluation	13 / 13 questions passed

KEY IDEA **Define once, use everywhere**

The semantic view is the **single source of truth**. The agent writes its SQL from it, and the dashboard marts are built from it. That is why the AI answer and the dashboard always show the same number.

2. Data architecture: the layers and why

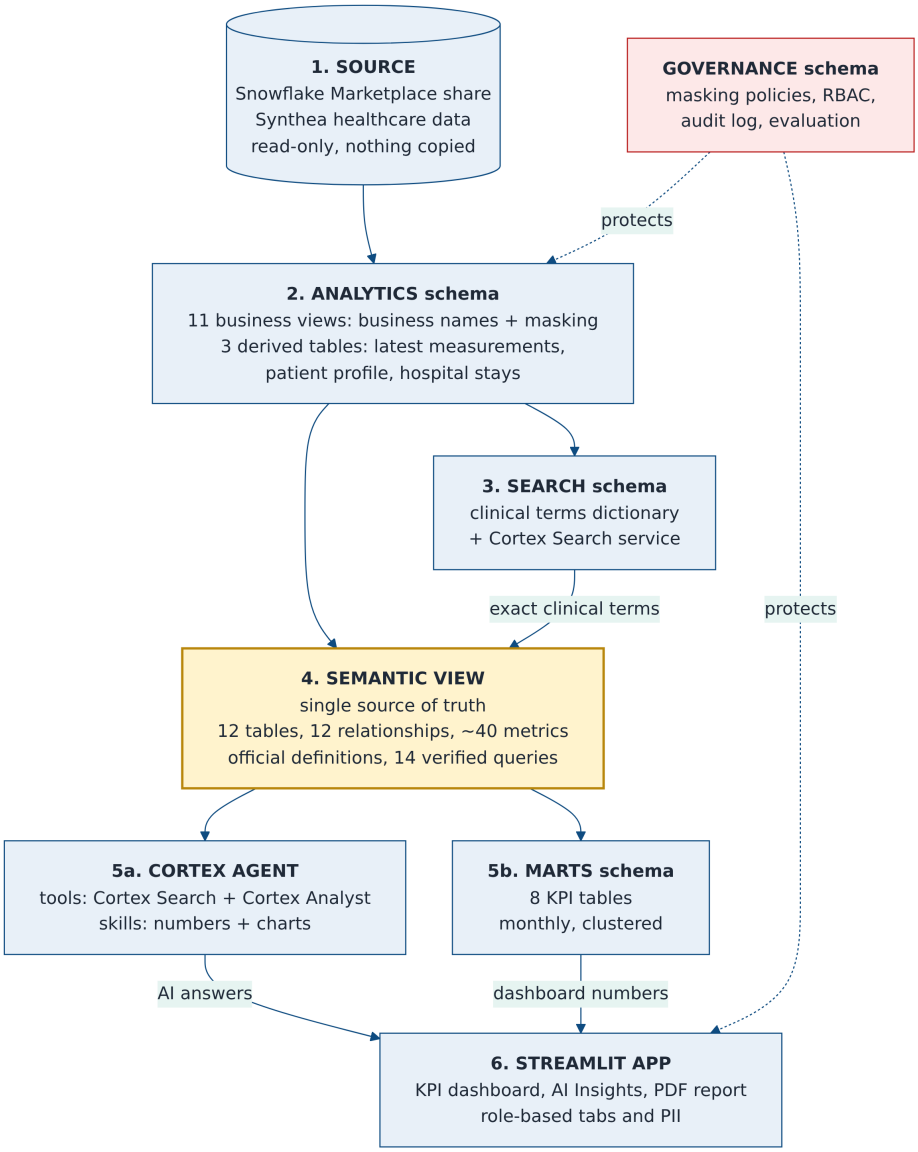


Figure 1. Data flows top to bottom. Governance protects the layers; the semantic view sits in the middle.

Each layer: what, where and why

Layer	Snowflake objects	What it does	Why this way
1. Source	Marketplace share	Raw Synthea tables (patients, encounters, conditions...)	A share is **read-only**: we can only SELECT, so we build in our own database
2. Business views	ANALYTICS: 11 views	Business names, simple flags (is_active...), masking	Views copy no data and are always up to date; this is the only place raw names appear
2. Derived tables	ANALYTICS: 3 objects	Latest measurement per patient, hospital stays + readmission flag	Heavy logic (763mn rows) is computed once and stored, so questions are fast
3. Search	SEARCH: table + Cortex Search	Dictionary of every clinical term	Bridges everyday words and clinical vocabulary

Layer	Snowflake objects	What it does	Why this way
4. Semantic view	ANALYTICS: semantic view	Business meaning of the data	One place for definitions, joins and metrics
5a. Agent	ANALYTICS: agent	Plans, calls tools, answers	Natural-language access for everyone
5b. Marts	MARTS: 8 tables	Pre-aggregated KPIs by month	Dashboard loads in seconds and costs little
6. App	Streamlit in Snowflake	Dashboard, AI chat, report	Data never leaves Snowflake
Governance	GOVERNANCE schema	Policies, permissions, logs, evaluation	Security and audit in one place

Why not a medallion (bronze / silver / gold) pipeline?

The Marketplace data is already clean (the provider calls it *SILVER*). We did not need to ingest or clean anything. Our work is the **serving and semantic layer** on top: business naming, definitions, security, AI and the app. If you are asked, say: "*the source was already curated, so I focused on the semantic layer and the consumption layer, not on ingestion.*"

Views or tables?

Use a VIEW when...	Use a TABLE when...
You only rename, filter or mask columns	The logic is heavy and the same every time (e.g. 763mn rows to 1 per patient)
You want data always fresh with no copy	You need speed for dashboards (marts)
The source is a share (no storage cost)	You want clustering on a column

Config-driven build

Every object name lives in `config.py`. The SQL files use placeholders such as `{{DATABASE}}`, and `run.py` swaps them for the real names before running each statement. Changing a name means changing one line, not twenty files.

```
config.py   DATABASE = "HEALTHCARE_INTELLIGENCE"
SQL file    USE SCHEMA {{DATABASE}}.{{ANALYTICS}};
run.py      -> USE SCHEMA HEALTHCARE_INTELLIGENCE.ANALYTICS;
```

3. The build, file by file

File	What it creates	Why
01_setup	Warehouse, database, 4 schemas, 5 roles, role hierarchy	Foundation; you can switch roles to test
02_business_views	11 business views, role-based masking policies, PII function for the app	Business names + privacy at the lowest layer
03_derived_tables	PATIENT_LATEST_MEASUREMENTS, PATIENT_PROFILE, HOSPITAL_STAYS	Fast answers for measurements and readmissions
04_search	CLINICAL_TERMS + Cortex Search service	Vocabulary bridge for the agent
05_semantic_view	The semantic view (12 tables, ~40 metrics, 14 verified queries)	The brain of the platform
06_agent	Agent in SQL (we built ours in the UI instead)	Optional: agent as code
07_marts	8 KPI marts, monthly, clustered	Fast dashboard with the same numbers as the AI
08_evaluation	Gold questions, RUN_AGENT_EVAL, scorecard	Prove the agent is correct (13/13)
09_audit_log	Event copy table, daily task, conversation history view	Who asked what, time and tokens
10_security	Grants, tab permissions, PII levels, app log tables	Who sees what
11_snowflake_intelligence	Adds the agent to Snowflake's own chat screen	Optional (we used the UI button)

WATCH OUT Run order rules

After re-running **05** (semantic view), also run **07** (marts) and **10** (grants): CREATE OR REPLACE removes the old grants.

Never run **06** when your agent was built in the UI: it would replace the UI agent with the SQL version.

Never write `CREATE OR REPLACE STAGE` for the skills stage: it creates a new, empty stage and deletes your uploaded skills. Use `CREATE STAGE IF NOT EXISTS`.

Run only the files you changed, for example `run.run(["09_audit_log.sql", "10_security.sql"])`.

4. Semantic modelling, model, view and layer

These four words sound alike but mean different things. Think of writing a dictionary for your company.

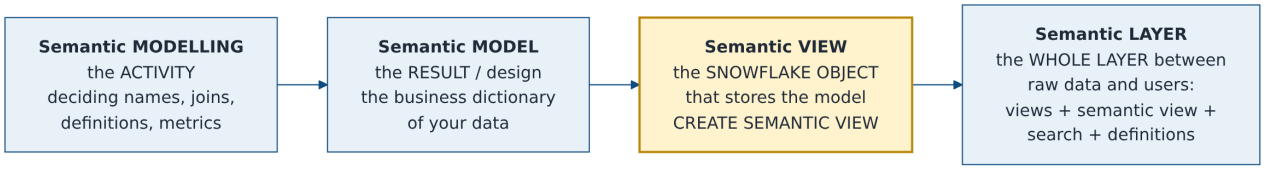


Figure 2. From an activity, to a design, to a Snowflake object, to a whole layer.

Term	Plain meaning	Dictionary analogy	In our project
Semantic modelling	The **activity** of describing data in business terms	Writing the dictionary	Deciding names, joins, the heart-attack definition, metrics
Semantic model	The **result**: the business description itself	The finished dictionary	12 tables, 12 relationships, ~40 metrics, definitions

Term	Plain meaning	Dictionary analogy	In our project
Semantic view	The **Snowflake object** that stores the model	The dictionary printed and put on the shelf	HEALTHCARE_SEMANTIC_MODEL (file 05)
Semantic layer	**Everything** between raw data and the people using it	The library: dictionary + index + rules	Business views + semantic view + Cortex Search + marts

IN OUR PROJECT Older vs newer way

Cortex Analyst used to read a **YAML file** on a stage (a "semantic model file"). The newer way is a native **semantic view** object created with `CREATE SEMANTIC VIEW`. It has grants, can be queried with `SEMANTIC_VIEW()`, and lives with the data. We used the semantic view.

5. The semantic view in detail

A semantic view tells Snowflake (and the AI) **what the data means**. Each part answers one question:

Part	Answers the question	Example from our project
TABLES	Which tables, with which key and business name?	patients AS PATIENT_PROFILE PRIMARY KEY (PATIENT_NUMBER), synonyms 'people', 'members'
RELATIONSHIPS	How do tables join?	visits (PATIENT_NUMBER) REFERENCES patients
FACTS	Which row-level numbers exist?	visits.claim_value = claim amount of ONE visit
DIMENSIONS	What can I group or filter by?	age_group, care_setting, visit_month, is_heart_attack
METRICS	What are the official calculations?	patient_count = COUNT(DISTINCT PATIENT_NUMBER)
SYNONYMS	What other words do people use?	'MI', 'AMI', 'STEMI' for heart attack
COMMENTS	What does it mean, and how to use it?	'OFFICIAL heart attack definition... ALWAYS use it'
SAMPLE_VALUES / IS_ENUM	Which values exist?	care_setting: inpatient, emergency, wellness...
CORTEX SEARCH link	Where to find exact free-text values?	diagnosis_name WITH CORTEX SEARCH SERVICE ... USING CLINICAL_TERM
VERIFIED QUERIES	What is the proven SQL for key questions?	14 questions, e.g. heart attack count
AI_SQL_GENERATION	Extra rules when writing SQL	'Count people with COUNT DISTINCT', 'rates x 100'
AI_QUESTION_CATEGORIZATION	What to refuse?	Requests for names, SSNs or individual patients

Fact, dimension or metric? A simple test

Type	Question to ask	Example
Fact	Is it a number about ONE row?	Claim amount of one visit: $120.50
Dimension	Would I say "BY ..." or "WHERE ..."?	BY care setting, WHERE year = 2023
Metric	Is it a total, count, average or rate over many rows?	Total claim amount, readmission rate

Official definitions as flags

Instead of hoping the AI picks the right diagnoses, we wrote the definition once as a yes/no dimension:

```
diagnoses.is_heart_attack AS DIAGNOSIS_NAME IN (
    'Myocardial infarction (disorder)',
    'Acute ST segment elevation myocardial infarction (disorder)', ...)
  WITH SYNONYMS = ('heart attack', 'MI', 'AMI', 'STEMI', 'NSTEMI')
  COMMENT = 'OFFICIAL heart attack definition ... ALWAYS use it'
```

We did the same for **blood pressure medication**, **flu / COVID vaccines**, **open claims** and **30-day readmission**. Everyone (AI, dashboard, analyst) now uses the same definition.

IN OUR PROJECT Why our SQL semantic view beat the quick ones

Question: "*How many patients have had a heart attack?*"

Autopilot (UI, generated): **refused**. CoCo-generated view: **0**, because it filtered the text 'heart attack', which does not exist in the data. Our hand-built view: **61,820**, correct.

Reason: the official definition flag + synonyms + Cortex Search link + a verified query. Generated models are a good start, but business definitions must be added by a person who knows the data.

Semantic view checklist: important points

#	Point	Why it matters
1	**Only one join path** between any two tables	Two paths make the join ambiguous; Cortex Analyst may pick the wrong one or fail
2	Primary keys must be **really unique**	Otherwise counts double after a join
3	Count people with **COUNT(DISTINCT ...)**	A patient with 5 visits is still 1 patient
4	Write **official definitions** as flags, with synonyms and comments	The AI follows written rules; it cannot guess your business rules
5	Use **business names and good comments**	The AI reads names and comments to choose columns
6	Link **Cortex Search** to free-text columns with many values	Diagnosis, medication, allergen names: the AI finds exact values
7	Use **SAMPLE_VALUES / IS_ENUM** for short lists	e.g. care settings: the AI knows the valid values
8	Add **verified queries** for important and tricky questions	They guide the AI toward proven SQL
9	**No personal data** in the model	Our model uses PATIENT_PROFILE, which has no names or SSNs
10	Be explicit about meaning (e.g. **latest** BMI, not history)	Prevents wrong trend answers; the agent says trends are not available

#	Point	Why it matters
11	Test with **tricky questions** in the playground	Synonyms, two-table questions, refusals
12	After CREATE OR REPLACE, **re-grant** access	Replacing removes grants (run file 10)

Three ways to create a semantic view

Way	Good for	Watch out
SQL (CREATE SEMANTIC VIEW)	Full control, version control in Git, repeatable	More typing; you must know the syntax
UI: Autopilot	Fast start from selected tables and example SQL	Add relationships, definitions and synonyms yourself
CoCo (Cortex Code assistant)	Drafting from a prompt	Check every definition; it may not know your vocabulary

6. Cortex Search

What it is: a managed search service. You give it a text column, and it finds the best matching rows for a query, using both meaning (semantic search) and keywords. It refreshes itself on a schedule (TARGET_LAG).

Why we need it: the vocabulary gap. The user types "peanuts" or "heart attack"; the data holds "Peanut (substance)" and "Myocardial infarction (disorder)". SQL needs the exact value.

How we built it

```
CLINICAL_TERMS  = every DISTINCT term from 8 sources (diagnoses, medications, procedures,
                  measurements, vaccines, allergens, visit types, visit reasons) + TERM_TYPE
CREATE CORTEX SEARCH SERVICE CLINICAL_TERMS_SEARCH
  ON CLINICAL_TERM  ATTRIBUTES TERM_TYPE  TARGET_LAG = '1 day'
  AS (SELECT CLINICAL_TERM, TERM_TYPE FROM CLINICAL_TERMS);
```

Important point	Why
Index the **vocabulary**, not the 763mn rows	One row per distinct term: small, fast and cheap
Add an **attribute** such as TERM_TYPE	Lets you filter: only diagnoses, only medications
Link it in the semantic view (WITH CORTEX SEARCH SERVICE)	Cortex Analyst uses it to find exact values
Also give it to the agent as a tool, with a clear description	The agent can look terms up before writing SQL
TARGET_LAG sets freshness and cost	'1 day' is enough for a slowly changing vocabulary
Grant USAGE on the service to the roles that ask questions	Without it, the tool fails for that role

7. Cortex Analyst

What it is: Snowflake's text-to-SQL service. It reads the **semantic view**, turns a question into SQL, and the warehouse runs that SQL. In our agent it is the tool called `healthcare_analyst`.

Important point	Why
Its answers are only as good as the **semantic view**	It works from names, comments, definitions and verified queries
It does not read all your data to answer	It sees the model, sample values and search results; the warehouse runs the SQL
It returns the **SQL** it used	Answers can be checked; our app shows the SQL under "How the agent answered"
It can **refuse** or say it cannot answer	Controlled by AI_QUESTION_CATEGORIZATION and your instructions
It is for **structured** data	For documents or free text, use Cortex Search
The SQL runs on a **warehouse**	That costs credits; derived tables keep queries small

KEY IDEA **Analyst vs Search vs Agent in one line each**

Cortex Search finds the right words. **Cortex Analyst** turns a question into SQL. **Cortex Agent** is the manager that decides when to use which, and writes the final answer.

8. Cortex Agent

What it is: an AI assistant that **plans**, **calls tools**, and **writes the answer**. We created ours in the UI as `HEALTHCARE_AGENT_UI`.

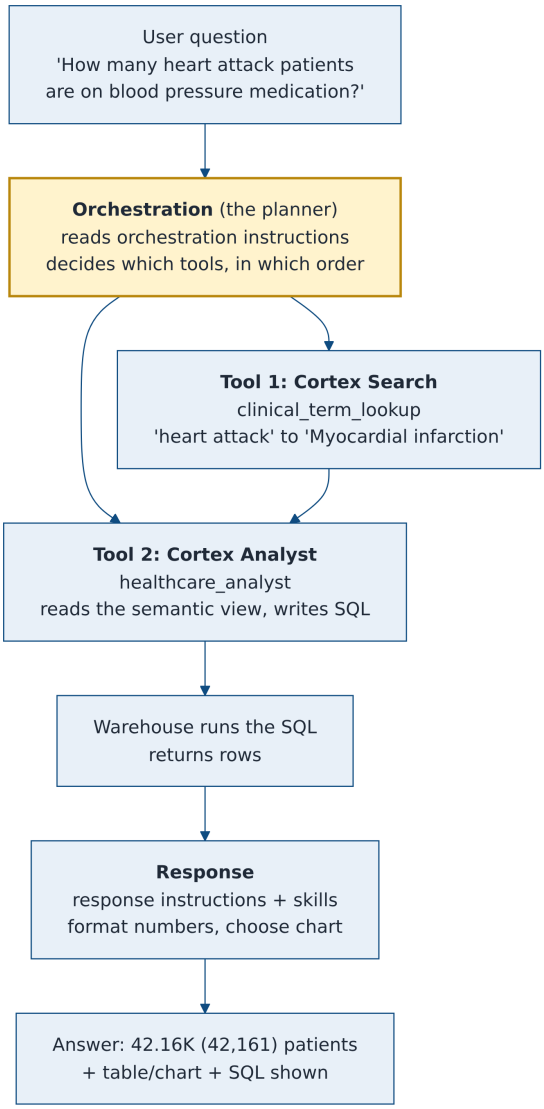


Figure 3. How the agent answers one question.

What an agent contains

Part	What it is	Our setting
Description	What the agent is for (shown to users)	Healthcare analytics assistant on synthetic data
Model	The LLM that plans and writes	auto (Snowflake picks)
Orchestration instructions	**How to work**: which tool when, rules to follow	Use search first for clinical terms; use official definitions; never give personal data
Response instructions	**How to answer**: tone, length, format	Lead with the number; formats via skills; end with the synthetic-data note
Tools	What it can use	clinical_term_lookup (Search), healthcare_analyst (Analyst)
Tool descriptions	When to use each tool	Clear text; the planner reads it to choose
Skills	Detailed rules loaded only when needed	number_formatting, chart_types, chart_styling
Budget	Time and token limits per question	120 seconds, 20,000 tokens

Part	What it is	Our setting
Sample questions	Starter questions shown to users	5 questions
Access	Who can use / monitor it	USAGE to DATA_ANALYST, MONITOR to DATA_ENGINEER
Publish	Draft vs live	Changes apply only after Publish

Where does each rule belong?

Kind of rule	Put it in	Example
Business meaning and definitions	Semantic view	What counts as a heart attack
Which tool, in which order	Orchestration instructions	Look up clinical terms before querying
Tone, length, structure of the answer	Response instructions	Lead with the key number, under 150 words
Long detailed how-to rules	Skills	K / mn / bn formats, chart choice and styling
Privacy refusal	Semantic view + orchestration	Refuse names, SSNs, individual records

Agent checklist: important points

#	Point	Why it matters
1	Keep instructions **short and clear**	They are sent with **every** question and cost tokens. One of our questions used 309,199 input tokens (seen in the monitoring traces).
2	**No conflicting rules**	We had "give exact counts" in instructions and "use K / mn" in a skill: the agent answered 1,421,656 instead of 1.42mn. Fixed by removing the old line.
3	Write **good tool descriptions**	The planner chooses tools by reading them
4	Put business logic in the **semantic view**, not in instructions	One place, used by agent and dashboard
5	Say clearly what to **refuse**	Personal data and individual patients
6	Set a **budget**	Stops long, expensive loops
7	Always **Publish** after editing	Draft changes are not live
8	**Evaluate** with test questions before sharing	Agents can be confidently wrong
9	**Monitor** traces: tools used, SQL, tokens, time	Find slow or wrong answers
10	Give **USAGE** only to roles that need it	The agent is a secured object
11	UI agent and SQL file: pick one source of truth	CREATE OR REPLACE AGENT overwrites the UI version

9. Agent skills

What a skill is: a folder with a `SKILL.md` file of detailed instructions. The agent loads a skill **only when it is relevant**. Instructions, by contrast, are sent with every question.

```
@AGENT_SKILLS stage
  skills/number_formatting/SKILL.md
  skills/chart_types/SKILL.md
  skills/chart_styling/SKILL.md

SKILL.md starts with:
---
name: number_formatting
description: Format every number - counts as K / mn / bn, money as $, percentages with 2 decimals...
---
# Number Formatting Rules ...
```

Important point	Why
Each skill is its own folder, and the file is always called **SKILL.md**	That is how the agent finds it
The **name** in the frontmatter matches the folder	Avoids confusion when adding the skill
The **description** decides when the skill loads	Write it as "use this when..."
The stage must use **server-side encryption** (SNOWFLAKE_SSE)	Client-side encrypted stages are not supported
One clear job per skill	Our 3 skills do not overlap: numbers, chart type, chart look
Never CREATE OR REPLACE the stage	That deletes the uploaded skills

Our formatting rules (from the mentor)

Type	Rule	Example
Counts	K / mn / bn with 2 decimals; exact count in brackets in the first sentence	61.82K (61,820)
Money	$ + K / mn / bn	$174.25bn
Percentages	Exactly 2 decimals	17.06%

10. Evaluation

Why: an agent can sound sure and still be wrong. Evaluation is an **exam**: fixed questions with known correct answers.

	UI Agent Evaluations	SQL harness (file 08)
Who marks the answer	An AI judge (LLM)	Rules: the right number must appear; PII must be refused
Expected answers	Typed into a dataset table	Calculated by SQL from the data
Dataset format	input_query (VARCHAR) + ground_truth (VARIANT: {"ground_truth_output": ...})	EVAL_GOLD_QUESTIONS with check type, number, tolerance
Result in our project	Judge was blocked on the account (error 403)	**13 / 13 passed**, shown in the app's Quality tab

	UI Agent Evaluations	SQL harness (file 08)
Runs automatically	Manual from the UI	Yes: CALL RUN_AGENT_EVAL()

KEY IDEA **Evaluation dataset vs verified queries**

Verified queries are part of the semantic view: they **teach** and guide the agent.

The **evaluation dataset** does not teach anything: it only **tests** the agent, like an exam paper.

Our 13 questions deliberately test hard cases: a synonym ("MI"), a two-table question (heart attack + BP medication), the vocabulary gap (peanuts), a question the data cannot answer (BMI trend: only the latest BMI is stored), and a privacy request that must be refused.

11. Security and governance

Roles (RBAC)

Five roles: **DATA_ENGINEER**, **DATA_ANALYST**, and three app personas (**HC_EXECUTIVE**, **HC_CLINICAL_ANALYST**, **HC_FINANCE_ANALYST**). The personas and the engineer inherit the analyst role, so the basic grants are written once.

Masking: Standard vs Enterprise

	Standard edition (what we did first)	Enterprise edition (what we use now)
How	IFF(CURRENT_ROLE() = ..., value, masked) inside a secure view	MASKING POLICY objects attached to columns
Where the rule lives	Inside the view	Separate object in GOVERNANCE, reused on many columns
Change a rule	Rewrite the view	Change the policy once
Error on Standard	-	"Unsupported feature 'MASKING POLICY'"

Column	DATA_ENGINEER	HC_CLINICAL_ANALYST	All other roles
First / last name	real	real	***MASKED***
SSN	real	XXX-XX-1234	XXX-XX-XXXX
Street address	real	***MASKED***	***MASKED***
Birth / death date	real	real	year only (1 Jan)

Role-based PII inside the app

A Streamlit app runs with its **owner's rights**, so inside the app CURRENT_ROLE() is always the owner's role, not the role picked in the dropdown. A masking policy therefore cannot tell app users apart. Our fix: the app looks up the selected role's PII level (FULL / PARTIAL / NONE) in APP_ROLE_PERMISSIONS and calls a **secure function**, `PATIENTS_WITH_PII(level)`, that applies that level.

KEY IDEA Two layers, one rule

Data layer: masking policies protect anyone querying in a worksheet or tool.

App layer: the PII function + tab permissions protect app users. Both follow the same table of who sees what.

Other protections

Protection	How
AI_REDACT	Removes names, SSNs and addresses from every question before the agent sees it
No PII in the semantic view	The model uses PATIENT_PROFILE, which has no identifiers
Refusal rules	AI_QUESTION_CATEGORIZATION + orchestration instructions
Deny by default	A tab is hidden unless APP_ROLE_PERMISSIONS has a row for that role
Audit log	Daily task copies agent events; view shows who, when, question, answer, time, tokens

REMEMBER Interview detail: CURRENT_ROLE vs IS_ROLE_IN_SESSION

We used CURRENT_ROLE() so each role's demo is clear. IS_ROLE_IN_SESSION() also counts inherited roles, which is often preferred in production (e.g. a manager role that includes the engineer role).

12. Performance: marts, clustering, caching

Technique	What we did	Honest note
KPI marts	8 small tables built FROM the semantic view with SEMANTIC_VIEW()	The dashboard reads thousands of rows, not millions, and matches the AI exactly
Clustering	HOSPITAL_STAYS by admission date; monthly marts by month	Clustering helps most on very large tables. Our marts are small, so here it mainly shows the pattern
Result cache	Snowflake reuses results of identical queries	Free and automatic when data has not changed
App cache	st.cache_data(ttl=600) on every query	Repeated clicks do not re-run SQL for 10 minutes
Warehouse sizing	MEDIUM for heavy builds, back to XSMALL; auto-suspend 60 s	Pay for size only while it is needed
Derived tables	763mn measurements reduced to 1 row per patient	The biggest speed win in the project

13. The Streamlit app

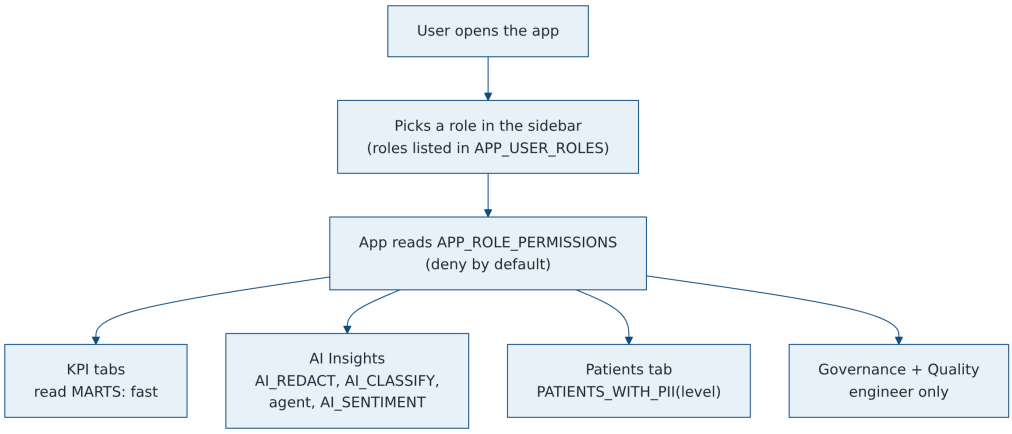


Figure 4. The role decides which tabs and how much personal data the user sees.

Tab	What it shows	Data from
Overview	Headline KPIs, visits and cost trends, AI summary button	Marts + AI_COMPLETE
Clinical	BMI, blood pressure, HbA1c categories; top diagnoses and allergies	Marts
Hospital	Admissions, readmission rate by period and age group, length of stay	Marts
Pharmacy & Vaccines	Top medications, cost, flu / COVID doses	Marts
Claims & Cost	Claims, open claims, claim amount by insurer	Marts
Patients	Patient lookup with role-based PII	PATIENTS_WITH_PII function
AI Insights	5-question dropdown, search bar, 7-day history, agent answers with chart and SQL	Agent
Governance	Audit log, questions by topic, feedback sentiment, masking demo	GOVERNANCE tables
Quality	Evaluation scorecard (13 / 13)	EVAL_SCORECARD

AI functions used in the app

Function	Job in the app
AI_REDACT	Cleans personal data out of every question and feedback text
AI_CLASSIFY	Tags each question with a topic (Clinical, Hospital, ...) for the Governance chart
AI_SENTIMENT	Reads user feedback on answers: positive, negative, neutral or mixed
AI_COMPLETE	Writes a 3-bullet summary of the headline numbers (cached to save credits)
DATA_AGENT_RUN	Sends the conversation to the agent and returns its full answer

App checklist: important points

Point	Why
The app runs with the **owner's rights**	Masking by CURRENT_ROLE() does not see the dropdown role; use the PII function
Use the **warehouse runtime** + environment.yml	The container runtime asks for pyproject.toml (the error we saw)
Add packages once (e.g. **reportlab** for the PDF report)	Saved in environment.yml; needed again only for a new app
The app needs its own copy of **config.py**	A UI-created app cannot see your Workspace files
Read **marts**, not raw views, for charts	Fast and cheap
Cache queries with **st.cache_data**	Fewer warehouse runs
Show business headers and formatted numbers	No raw column names; K / mn / bn and 2-decimal %
Log questions **already redacted**	The log itself never stores personal data
Hide tabs by default; show only allowed ones	Users never see what they should not

14. Problems we hit and how we fixed them

Problem	Cause	Fix
Could not create views in the Marketplace database	A share is read-only	Build everything in our own database
"Unsupported feature 'MASKING POLICY'"	Account was Standard (upgrade not yet active)	Secure view with IFF on Standard; masking policies once Enterprise
"Client-side encryption stages are not supported"	Skills stage created without SSE	CREATE STAGE ... ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE')
Skills not recognised	Files named SKILL_style.md, wrong folders	One folder per skill, each with SKILL.md
"pyproject.toml file does not exist"	App created on the container runtime	RUNTIME_NAME = 'SYSTEM$WAREHOUSE_RUNTIME' + environment.yml
Agent showed 1,421,656 not 1.42mn	Instruction conflicted with the skill	Removed the old instruction line
UI evaluation: ground truth error	ground_truth was an OBJECT	Cast to ::VARIANT
UI evaluation judge: 403	Account not allowed to use the judge endpoint	Used the SQL harness (13/13)
AI_COMPLETE "not available for trial accounts"	Trial accounts block some LLM functions	Used the paid account
Governance tab: "cannot convert float NaN"	Empty audit log	COALESCE totals + an info message
CoCo semantic view answered 0	Filtered the text 'heart attack'	Official definition flag + search link
Risk: skills deleted on re-run	CREATE OR REPLACE STAGE in a build file	CREATE STAGE IF NOT EXISTS

15. Interview quick answers

Q: What is a semantic view?

A Snowflake object that stores the business meaning of data: tables, relationships, facts, dimensions and metrics, plus synonyms, comments and verified queries. Cortex Analyst uses it to write correct SQL.

Q: Semantic layer vs semantic view?

The semantic view is one object. The semantic layer is everything between raw data and users: business views, the semantic view, search and marts.

Q: How do you stop the AI giving different answers to the same question?

Official definitions as flags in the semantic view, verified queries, and building the dashboard marts from the same semantic view.

Q: How does the agent handle 'heart attack' when the data says 'Myocardial infarction'?

Cortex Search finds the exact clinical term, and the semantic view has a heart-attack flag with synonyms.

Q: Why skills instead of long instructions?

Instructions are sent with every question and cost tokens; skills load only when relevant.

Q: How did you protect personal data?

Role-based masking policies, no PII in the semantic model, AI_REDACT on questions, refusal rules, and a PII function in the app because Streamlit runs with owner's rights.

Q: How did you know the agent was correct?

13 gold questions with answers calculated by SQL, checked by rules: 13/13 passed. Plus UI Agent Evaluations.

Q: Why marts if you have the semantic view?

Speed and cost for the dashboard. They are built from the semantic view, so the numbers match the AI.

16. Glossary

Term	Meaning
Share (Marketplace)	Read-only data from another Snowflake account; nothing is copied
Secure view	A view whose definition is hidden from users who only query it
Masking policy	A rule that changes what a column shows depending on the role (Enterprise+)
RBAC	Role-based access control: permissions are given to roles, roles to users
Semantic view	Snowflake object holding the business meaning of data
Fact	A number about one row (claim amount of one visit)
Dimension	Something to group or filter by (care setting, month)
Metric	An aggregation over many rows (patient count, readmission rate)
Verified query	A question + proven SQL stored in the semantic view to guide the AI

Term	Meaning
Cortex Search	Managed search over a text column (meaning + keywords)
Cortex Analyst	Text-to-SQL service that reads the semantic view
Cortex Agent	AI that plans, calls tools (Search, Analyst) and writes the answer
Orchestration	The agent's planning step: which tool, in which order
Skill	A SKILL.md file of detailed instructions the agent loads only when relevant
Budget	Time and token limits for one agent answer
Token	A small piece of text; LLM cost and limits are counted in tokens
Mart	Small pre-aggregated table for dashboards
Clustering	Organising a table by a column so queries can skip data (pruning)
Owner's rights	The app runs with its owner's privileges, not the viewer's
CoCo	Cortex Code, the AI coding assistant inside Snowsight
Snowflake CoWork	Snowflake's own chat screen for agents (formerly Snowflake Intelligence)